Specification Universal Control Room Interface (UCRI2) Version 2.0.0

Table of Contents

1. Introduction
2. System Architecture
    2.1. Architecture Style Messaging
    2.2. Switching Layer
    2.3. Application layer
    2.4. Security concept
    2.5. Addressing concept
        2.5.1. Exemplary OID hierarchy
        2.5.2. Example OID nomenclature
    2.6. Versioning
        2.6.1. Transport layer versioning
        2.6.2. App versioning
3. Communication protocol
    3.1. Transmission of messages
    3.2. Message validation
    3.3. KT register
    3.4. Data integrity
        3.4.1. Signing
        3.4.2. Encryption
4. API
    4.1. Client API
        4.1.1. /token - HTTP GET
        4.1.2. /info - HTTP GET
        4.1.3. /registry - HTTP GET
        4.1.4. /registry/{id} - HTTP GET
        4.1.5. /messaging/send - HTTP POST
        4.1.6. /messaging/receive - HTTP POST
        4.1.7. /messaging/commit - HTTP POST
    4.2. Peer-to-Peer API (P2P API)
        4.2.1. /token - HTTP GET
        4.2.2. /info - HTTP GET
        4.2.3. /registry - HTTP GET
        4.2.4. /messaging/send - HTTP POST
    4.3. Error handling
        4.3.1. Error codes for responses with HTTP status code 400
        4.3.2. Error codes for responses with HTTP status code 401
        4.3.3. Error codes for responses with HTTP status code 500
5. Applications
6. System integration
    6.1. Integration patterns
    6.2. Integration scenarios

1. Introduction

UCRI stands for Universal Control Room Interface – a protocol for communication between two or more operations control systems.

Following its successful introduction and extensive testing in practice, experience was gained and new requirements identified. These necessitated further development of the UCRI protocol on a new architectural basis. UCRI2 is a completely revised version that supports all use cases of the previous version (UCRI 1.1) and forms the basis for future, flexible extensions.

The goals for the development of UCRI2 are:

- Simple, quick-to-implement API
  - Standardized machine-readable specification
  - Secure delivery and processing of messages
- Separation of technical and functional aspects
  - Easy expandability
  - Technical components do not need to be adapted for functional extensions
  - Functional extensions can be made without adjustments to the infrastructure
- Messaging-based architecture
  - Easy interaction between multiple manufacturers
  - Support for centralized and decentralized connections
- Consistent standardization
  - BSI-compliant network architecture and design
  - OpenAPI specifications for client and P2P APIs
  - JSON Schema specifications for applications (UCRI2 apps) for message validation
  - OAuth 2.0 for authentication and authorization
  - RFC 3447 – Cryptographic procedures
  - RFC 8785 – JSON Canonicalization Scheme (JCS)
  - RFC 7517 – JSON Web Key

This specification is divided into the following chapters:

System Architecture

This chapter describes UCRI2 as an n:m communication architecture between different communication participants using the messaging architecture style. The separation between the business application layer and the technical mediation layer as well as a comprehensive security concept are introduced.

Communication Protocol

The UCRI2 communication protocol defines two interfaces: the client API for communication between a control center system and the UCRI2 infrastructure, and the P2P API for communication between distributed components of the UCRI2 infrastructure based on peer-to-peer topology. This chapter describes mechanisms for message transmission and validation, as well as for ensuring data integrity through cryptographic methods.

API

The UCRI2 APIs are designed as REST services and documented with OpenAPI 3.1.0 specifications. This chapter describes the structure of the APIs in functional areas and the semantics of the individual API endpoints.

Applications

The description of the UCRI2 applications is provided separately from in the form of JSON schemas with accompanying documentation.

System integration

An important goal of the UCRI2 specification is to ensure interoperability in complex IT landscapes. This chapter uses various integration scenarios to explain how specific project requirements can be implemented with the help of specialized UCRI2 software components.

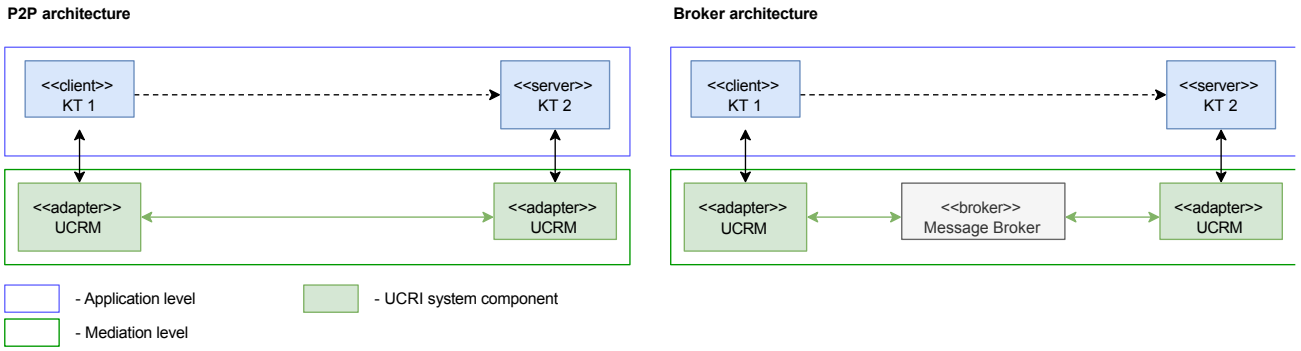
2. System architecture

In contrast to UCRI Version 1, which specifies 1:1 communication between control centers, UCRI 2 has been fundamentally developed for n:m communication between different participants.

2.1. Architectural style Messaging

The messaging architecture style is used to structure the UCRI2 interface. In this architecture style, distributed independent system components (generally referred to as communication participants, or CPs for short) communicate with each other using messages.

Messaging systems separate the business application (usually structured according to the client-server principle) from the communication layer—i.e., from the technical aspects of message transmission between the KT's technical systems. Messages can be converted during transmission without the sender or recipient being aware of the conversion. This decoupling enables integrators to implement different communication topologies depending on requirements, from decentralized P2P protocols to centralized broker architectures:



Messaging systems enable components to remain decoupled and focus on their own tasks, while still being able to communicate and collaborate with other components in the system. It reduces the number of dependencies between components, making the system more flexible and easier to maintain.

2.2. Routing layer

The other important architectural pattern used in structuring the UCRI2 interface is the adapter pattern. In this pattern, communication between the technical system of the KT (application level) and the switching level takes place via an adapter component (UCRI Control Room Module - UCRM). The adapter enables bidirectional communication between the levels in a standardized form and reduces the complexity of the connected interfaces.

The UCRM provides the UCRI2 client API – the only communication interface for directly connected communication participants such as control center systems or other technical nodes, as well as other external systems.

The UCRM adapters, which communicate with each other directly or via a broker, form the switching level and thus take care of the technical aspects of communication, while the KTs focus on the actual application logic—i.e., the application level.

The central task of the switching level is to deliver messages between sender and receiver. In addition, various cross-sectional tasks are performed at the switching level.

Individual tasks of the switching level are:

- Management of the communication topology, including addressing concept, KT status monitoring, and KT register
- Authentication, authorization, accounting
- Transmission of messages using the UCRI addressing concept
- Validation of application messages

The switching layer is free of technical expertise. It only implements data transport and ensures data integrity.

2.3. Application layer

The UCRI2 application layer is fundamentally separate from the switching layer. The application layer is divided into several independent applications, known as UCRI2 apps.

A UCRI2 application is defined by the following artifacts:

- A set of standardized message schemas (JSON, canonical data model)
- Process model (defined sequence of messages)
- Process definitions (specifications regarding application logic that must be taken into account during implementation in technical KT systems)

Individual applications are versioned separately and independently of the specification of the mediation layer (transport layer specification) (see UCRI2 versioning). This allows for the free further development of each individual UCRI2 app.

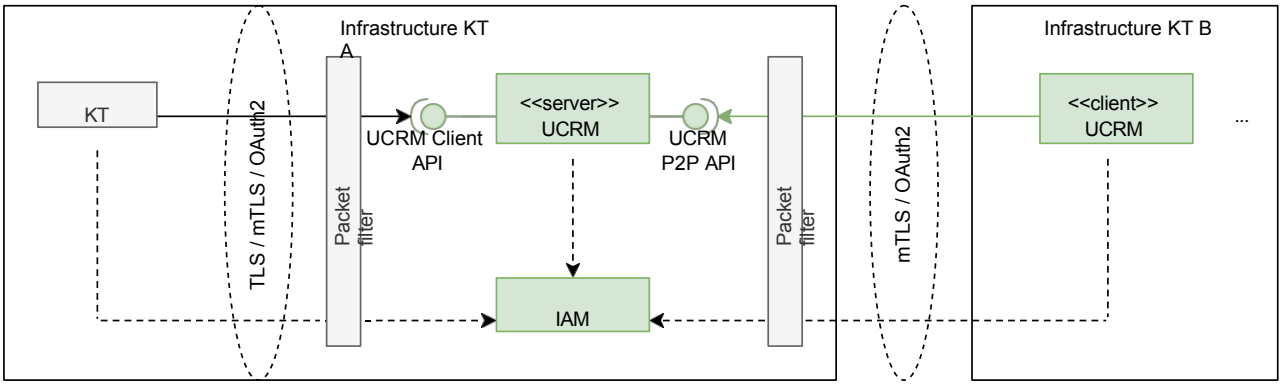
2.4. Security concept

Individual security aspects are considered below with regard to the standard UCRI2 communication architecture. Specific system solutions (see chapter System Integration) should take into account corresponding project-specific security requirements.

UCRI2 distinguishes between two communication domains, see figure:

- Communication between a control center system (KT system) and a UCRI control center module (UCRM). This communication takes place using the UCRI2 client API and typically occurs within an infrastructure controlled by a control center operator.
- Communication between two UCRMs. Inter-UCRM communication in a P2P architecture uses the UCRI2 P2P API. Inter-UCRM communication can take place via the public Internet and, in this case, requires special security measures.

P2P architecture



The infrastructure of a KT should have a P-A-P structure consisting of packet filters as a separation to trusted internal systems (control center), application layer gateway (UCRM), and packet filters as a separation to untrusted networks (Internet) (see Network Architecture and Design - BSI).

Both API implementations use OAuth 2.0 with client credentials grant type in both communication domains for secure authentication and authorization of clients at the application level.

Trust concept

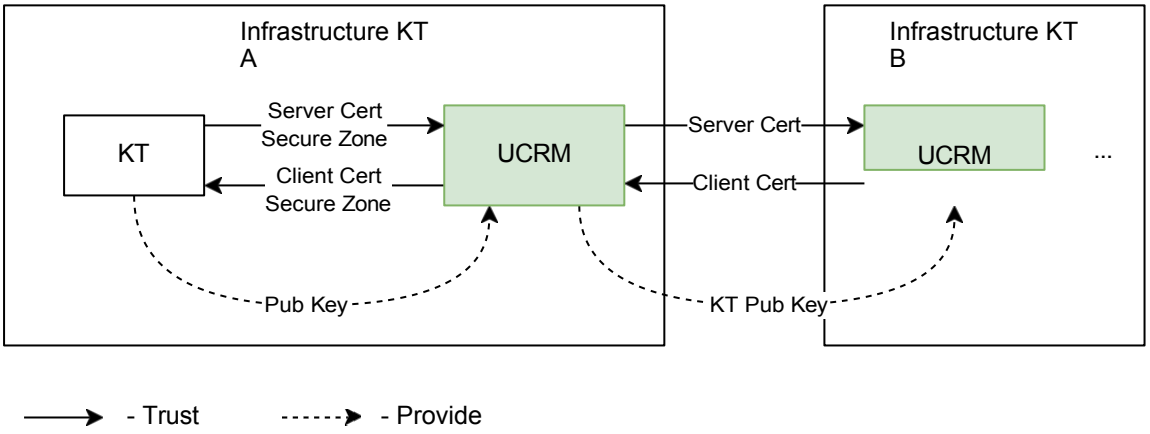
In a P2P architecture, mTLS is used to secure inter-UCRM communication between two UCRMs. Both the client and the server establish a mutual trust relationship via digital certificates. The packet filter on the untrusted network side can terminate the mTLS connection. A list of approved domain names can be implemented in the packet filter (white listing), thus restricting communication to a closed group of participants.

Communication between a KT system and the UCRM can be carried out either via TLS or mTLS, depending on local security requirements.

The trust relationships between the system components in client/server communication are based on the following mechanisms (see figure):

1. Server authentication. The trust relationship with the UCRM as a server is based on the domain name (DNS) of the server and is secured by a server certificate. The domain name of the server is anchored in the server certificate.
2. Client authentication. The trust relationship with the KT or UCRM in the client role is based on the object identifier (OID) of the client (see addressing concept) and is secured by a client certificate. The OID of the client is anchored in the client certificate.
3. Both the client and server have a list of trusted certification authorities (CAs), known as root CAs, which serve as trust anchors. The certificates issued by the client and server must be signed by a CA that is included in the other party's trusted list.

P2P architecture



Note 1: In specific customer situations, different certificate management mechanisms can be implemented—depending on requirements, either at a central authority or based on specific agreements between individual KTs.

Note 2: Alternatively, trust between KT systems and the UCRM can be established in an infrastructure controlled by a control center operator by creating a security zone at the network level.

To guarantee the secure origin and authenticity of application messages, the messages can be signed by the sender KT. To validate the signature, the KT creates a cryptographic key pair and provides its public key to the local UCRM module via a secure channel. In inter-UCRM communication, the KT's public key is transmitted to the external UCRM via the UCRM P2P API and is then available to potential message recipients via the client API for signature validation. The signing of messages is described in detail in the chapter on communication protocols.

2.5. Addressing concept

Unique identifiers in accordance with the Object Identifier Standard (OID specifications ISO/IEC 9834, DIN 66334) are used to address the individual communication participants (KT) at the switching level. The following specifications apply to the definition of OIDs:

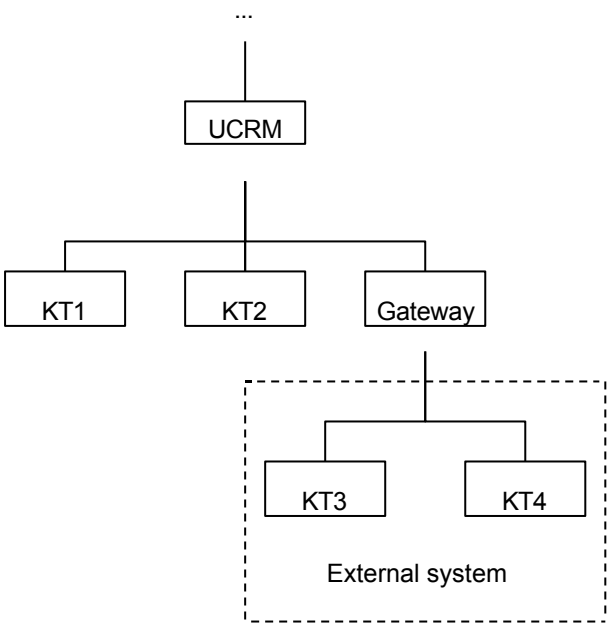
1. If possible, officially assigned OIDs should be used. These can be requested from the Federal Institute for Drugs and Medical Devices for the German healthcare system, for example.
2. If no officially assigned OIDs are used, self-assigned OIDs must NEVER be located in an address space in which officially assigned OIDs are also assigned, in order to avoid overlaps with officially assigned OIDs.

In addition, it is recommended that the OIDs used be reported to the UCRI expert forum so that address conflicts can be avoided directly during assignment.

The example OID hierarchies and OID nomenclature presented in the following subchapters are only recommendations. If OIDs assigned by official bodies are used, this does not necessarily result in a hierarchy as described in the following subchapters.

2.5.1. Example OID hierarchy

The addressing of individual KT can be organized hierarchically and then reflects the hierarchical communication structure:



The gateway system component represents a special KT. The gateway is used at the transition to external systems and provides a function for mapping between external source or destination addresses and internal OIDs. An external system is assigned a correspondingly reserved OID range (subtree) and the gateway receives the root OID address of this subtree.

2.5.2. Example OID nomenclature

All communication participants in a UCRI system are assigned a unique OID within that system. This OID address space is defined by a root address (hereinafter referred to as). The structure of the UCRI OID address space (UCRI OID nomenclature) enables KT to be addressed across national borders.

Each country is free to define its own addressing scheme below the country OID subtree. The root address of a country OID subtree is defined as follows:

- <Root OID>.1.<Country code> (example: .1.276 for Germany)

Each federal state in Germany is also free to define its own addressing scheme below the federal state OID subtree. The root address of a federal state OID subtree is defined as follows:

- <Root OID>.1.276.<State code> (Example: .1.276.5 for North Rhine-Westphalia)

Different organizations can also use different addressing schemes. The root address of an nPOLGA OID subtree is defined as follows:

- <Root OID>.1.276.5.<POL/nPOLGA code, etc.> (Example: .1.276.5.1 for nPOLGA in North Rhine-Westphalia)

The OID address of the individual communication participants (KT) is formed according to the official community code (AGS):

#	Level	Example
1	Root OID	1.2.3 - Specification in a closed UCRI system
2	Record type	1 - Single address, 2 - Group (currently, UCRI does not support group addresses)

3	Country code	276 - Germany according to ISO 3166
4	State code	5 - for North Rhine-Westphalia according to AGS
5	Identification of POL/nPOL hazard prevention (KRITIS, etc.)	Police stations, health authorities, veterinary offices, etc. Definition to follow. Assumption for example: 1 - nPOLGA, 99 - Test/Pilot
6	Code number of the administrative district	1 - Düsseldorf administrative district according to AGS
7	Code for the district or independent city ("0" for central institutions at the administrative district level)	58 - Mettmann district according to AGS
8	Municipality ("0" for independent cities)	28 - City of Ratingen, Ratingen district according to AGS
9	Control center module	1 - first control center module, consecutive numbering of control center modules
10	Communication participant	1 - e.g., ELS, consecutive numbering of KT

Example OID Fire and Rescue Service ELS in Ratingen: .1.276.5.1.1.58.28.1.1

2.6. Versioning

Versioning for the switching level (UCRI2 transport layer) and the UCRI2 apps is carried out separately. This specification

describes version 2.0.0 of the UCRI2 transport layer.

2.6.1. Transport layer versioning

The transport layer specification comprises this document and the OpenAPI specifications for the client and P2P interfaces.

The version number for the transport layer specification consists of three numerical parts:

`GEN.MAJOR.MINOR`

The following specifications apply to the three version components:

- `GEN` : Set to 2 for UCRI2
- `MAJOR`: Main version of the transport layer within the UCRI2 versioning. This version is changed when changes are made to the API endpoint definitions that either modify existing endpoints with regard to mandatory fields or add new mandatory fields.
- `MINOR` : Subversion of the transport layer. This version is changed when new optional fields are added to existing endpoints or new optional endpoints are added.

This means that changes to the `MINOR` version are always compatible, so that systems with matching `GEN.MAJOR` versions can communicate with each other, even if they have different `MINOR` versions.

transport layer messages

The transport layer uses a UCRI2 app called "Transport Layer Messages" (`transport_layer_messages`). This app describes messages that are created on the transport layer and consumed by connected UCRMs and clients. Therefore, a specific version of this app MUST be supported by all clients and UCRMs that implement version 2.0.0 of the transport layer.

The version of the transport layer messages app that must be supported for this purpose is 1.0.

2.6.2. App versioning

App versioning is done individually for each app. The version number of an app consists of two numerical parts:

`MAJOR.MINOR`

The following specifications apply to the two version components:

- `MAJOR`: Main version of the app. This version is changed when new mandatory messages are added or when mandatory fields are added or changed in existing messages.
- `MINOR` : Subversion of the app. This version is changed when new optional messages are added or optional fields are added to existing messages.

The UCRI2 protocol includes a KT register (see KT register) in which app versions supported by a communication participant supports. Although changes to the `MINOR`version are always compatible, `MINOR` -Variants must be explicitly specified under supported apps for a participant.

3. Communication protocol

The following section examines individual aspects of the UCRI2 communication protocol with regard to the standard UCRI2 communication architecture. Manufacturers of specialized UCRM modules (see chapter on system integration) must implement corresponding specific communication protocols.

From the perspective of a control center (KT system), UCRI2 is a client/server communication with the UCRI2 system component control center module (UCRM). A KT system (client) establishes a connection to the UCRM (server) and consumes the so-called **UCRI2 client API**. The UCRI2 client API is therefore the only interface with which control center software must be integrated in order to communicate with other control centers via UCRI2.

At the switching level, all UCRMs communicate directly with each other. Each UCRM must establish a direct network connection to each individual partner UCRM. At the same time, each UCRM offers an interface – the so-called **UCRI2 P2P API**, which is consumed by other partner UCRMs.

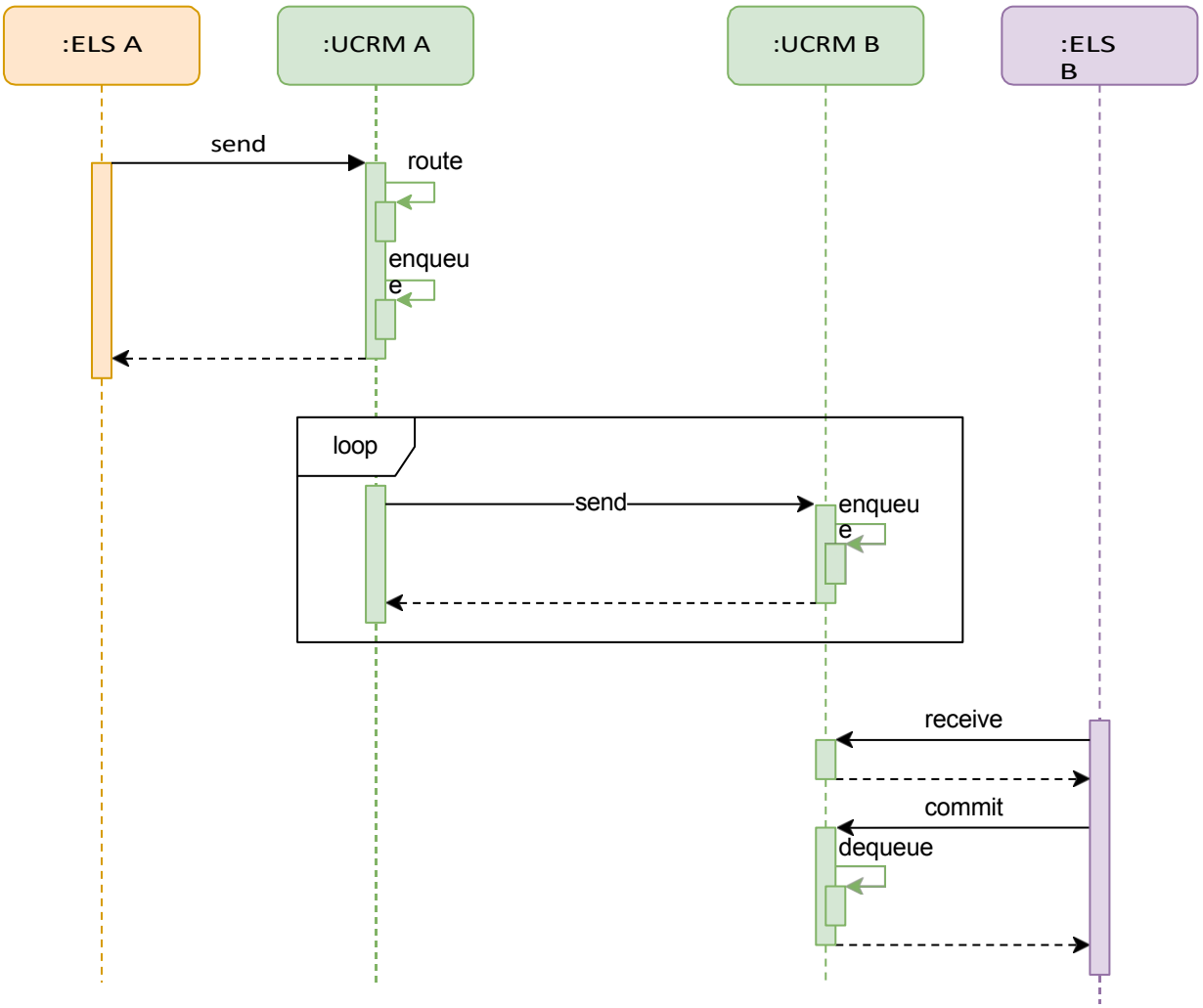
Depending on the role in the overall communication, this results in the following relevance of the present specification for the system components involved:

- A KT that wants to connect to other KTs via UCRI2 must implement the UCRI2 client API exclusively from the consumer perspective.
- A UCRM must implement the UCRI2 client API from the provider perspective. In addition, a UCRM that is involved in a communication system with P2P topology must implement the UCRI2 P2P API (provider and consumer perspectives).

The REST APIs Client API and P2P API are described in detail in the API chapter.

3.1. Transmission of messages

A KT system transfers outgoing messages to the UCRM module using the Client API endpoint **/send**. The messages are then forwarded by the local UCRM to the recipient UCRM using the P2P API endpoint **/send**. To handling the unavailability of partner UCRMs, an output buffer is implemented in the local UCRM.



3.2. Message validation

Messages are validated in UCRM using app-specific message schemas. Validation takes place synchronously during transmission. The KT register provides information about the application schemas supported by KT.

3.3. KT register

A KT registry is used to manage KTs. It contains system information (system name, operator, technical support, etc.), system status, and information about the supported UCRI2 applications of a communication participant. The KT registry is provided and kept up to date by the UCRM. The KT registry is available to the connected KT systems via the client API.

Each UCRM builds a local cache of the KT registry by regularly calling the P2P API endpoint **/registry** on all known partner UCRMs. In doing so, a UCRM only provides its own directly connected KTs.

A connected KT system queries the KT registry via the client API endpoint /registry. In doing so, a UCRM provides all KTs known to it (both its own and those determined via inter-UCRM communication).

Each UCRM has its own entry in the KT registry.

3.4. Data integrity

Checking data integrity in communication systems is crucial to ensure that the transmitted information originates from the correct sender and has not been manipulated during transmission.
cryptographic methods such as **signing**, the authenticity and immutability of the data is ensured, while **encryption** guarantees confidentiality and protects the content from unauthorized access.

The relevant technical guideline from the BSI (https://www.bsi.bund.de/SharedDocs/Downloads/DE/BSI/Publikationen/TechnischeRichtlinien/TR02102/BSI-TR-02102.pdf?__blob=publicationFile&v=9).

The asymmetric cryptographic RSA method RSASSA-PKCS1-v1_5 (RFC 3447: Public-Key Cryptography Standards (PKCS) #1: RSA Cryptography Specifications Version 2.1) is used both for encrypting and digitally signing messages .

The method uses a key pair generated for each KT, consisting of a private key and a public key. The private key is stored in the keystore of the KT system.

The public keys are exchanged and made available via the KT registry as part of the available KT data in the form of a JSON Web Key (JWK, RFC 7517: JSON Web Key (JWK)).

3.4.1. Signing

Message signatures ensure that a message has not been altered during transmission and that it originates from the specified source. This requires two steps: the generation of a unique hash value for the message and the signing of this hash value by the sending system:

- For hashing, the message is first converted into canonical form according to the JSON Canonicalization Scheme (JCS, RFC 8785: JSON Canonicalization Scheme (JCS)) and this canonical form is then hashed with SHA3-256.
- For signing and verifying the signature, the method according to the IETF standard JSON Web Signature (JWS, RFC 7517: JSON Web Key (JWK)) in the Compact JWS variant in combination with the RSA procedure RSASSA-PKCS1-v1_5 is used for signing and verifying the signature.

These steps are described in more detail below.

Hashing the messages

Before applying the JCS, the signing system creates a copy of the message that contains only the following fields:

- "source"
- "destinations"
- "payload"

This means that all other fields are not included in the hashing. This is necessary because other fields are optional or are set by the UCRM if they have not been set by the client. The hashing algorithm used is SHA3-256 (Use of the SHA3 One-way Hash Functions in the Cryptographic Message Syntax (CMS)).

Signing of messages

A header specifying the RSA-256 algorithm is used as the JWS header:

```
{
  "typ": "UCRI_PLAIN",
```

```
  "alg":
"RS256"
}
```

The calculated hash value of the message is used as the JWS payload. This is signed with the private key of the message sender. The result (digital signature) is transferred in the form of a JSON Web Signature (JWS, RFC 7517: JSON Web Key (JWK)) in the **signature** field:

```
BASE64(header) || '.' || BASE64(hash) || '.' || BASE64(signature)
```

Verification of signatures

To verify the transmitted JWS, the receiving system MUST in turn

- calculate the hash value of the received message
- Check the transmitted signature for validity
- Check that the signed hash value (JWS payload) matches the hash value of the received messages The hash

value is determined in accordance with "Hashing of messages".

The signature is extracted from the received message and validated against the sender's public key stored in the KT register.

Signing messages is optional. The KT register contains information in the **transmitsUnsignedMessages** field about whether a KT sends signed or unsigned messages.

3.4.2. Encryption

Since all communication between KT and UCRM and between UCRMs is TLS-encrypted, E2E encryption of the message content is not necessary. However, in order to meet stricter security requirements, especially for communication via the Internet, it may be advisable to additionally encrypt the message content end-to-end,
i.e., between the sender UCRM and the recipient UCRM or even between the KTs (in this case without the possibility of having the messages to be validated by the UCRM).

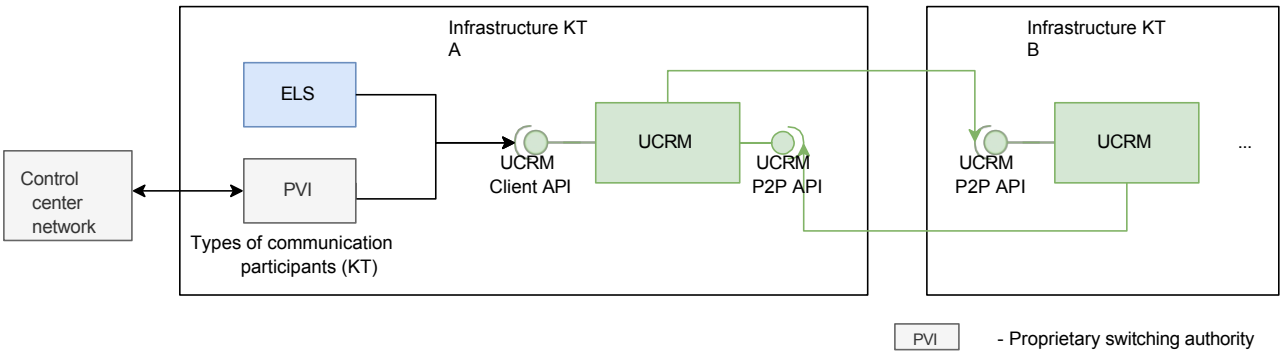
The RSA method RSASSA-PKCS1-v1_5 (RFC 3447: Public-Key Cryptography Standards (PKCS) #1: RSA Cryptography Specifications Version 2.1) is then used for encryption.

The specific encryption method will be specified in a later version of the UCRI2 standard if necessary.

4. API

This chapter describes the specific implementation of the UCRI2 APIs. According to the role concept, a distinction is made between two APIs, see figure:

- The **client API** is offered by **the UCRM** and consumed by **clients** (communication participants).
- The **P2P API** is offered by **UCRMs** that want to establish a connection with other **UCRMs** based on a peer-to-peer topology.



The following general specifications apply to both APIs and are supplemented by API-specific specifications in the respective subchapters:

1. The basic API design uses REST API design guidelines developed by TM Forum.
2. The specific design of the APIs, including the endpoints and data formats to be used, is defined in the OpenAPI specifications in version 3.1.0 (defined in the OpenAPI Initiative) belonging to the specification. The OpenAPI specifications are structured. The higher-level API descriptions (`ucrm-client.yaml` and `ucrm-p2p.yaml`) refer to the yaml files in the "schemas" subdirectory with the . In addition, there are bundled OpenAPI specifications with identical content in JSON format (`ucrm-client-bundled.json` and `ucrm-p2p-bundled.json`).
3. Both APIs are divided into the following functional areas:
   - Authentication via OAuth 2.0 Client Credential Grant Flow.
   - KT Registry - Querying properties of registered communication participants.
   - Messaging - Sending and receiving messages.
   - Info – Information about the version and operator of the interface.
4. Data transmission is carried out exclusively via transport encryption (TLS).
5. The error objects used to transmit error states (see `error.yaml`) are described in the subchapter Error Handling for both APIs, as most of the error codes used occur in both APIs .
6. The generation and verification of message signatures is described in the chapter Communication Protocol in the subchapter Data Integrity.
7. If a request does not comply with the OpenAPI specification, it MUST be rejected by the UCRM (see chapter Error Handling).
8. A UCRM that implements the **client API** MUST support it completely.
9. A UCRM that implements the **P2P API** MUST support it completely.

4.1. Client API

The Client API is used to connect KT (clients) to the transport layer (UCRM). The API endpoints are presented below:

4.1.1. /token - HTTP GET

The token endpoint is used to retrieve JSON Web Tokens (JWT), which are used for authorization at all other endpoints. For this purpose, a user ID and a secret are transferred via HTTP header. It should be noted that, according to RFC 7519, an exp claim is represented as a NumericDate (i.e., as the seconds since 1970-01-01T00:00:00Z UTC), see RFC 7519 Chapter 2.

4.1.2. /info - HTTP GET

The info endpoint can be used to retrieve general information about the UCRM, such as the UCRI2 transport layer version used, UCRM manufacturer, product version, and operating status of the UCRM.

4.1.3. /registry - HTTP GET

The registry endpoint allows KT to retrieve information about all communication participants known to the UCRM (including the UCRMs themselves). The data fields normally change only rarely, with the exception of the status field.
This indicates whether a KT is currently accessible. The value of this field changes depending on the connection status determined by the switching level

or the connection status determined by the UCRMs involved (see availability status under API endpoint

/messaging/receive. Since the environment is dynamic, i.e., new KTs can be added and existing KTs can be removed, KT registry queries should be performed regularly (at least once per hour, at most every 5 minutes).

4.1.4. /registry/{id} - HTTP GET

KTs can use the registry ID endpoint to retrieve information about a specific communication participant that has the transferred OID.

4.1.5. /messaging/send - HTTP POST

Messages can be sent to other KTs via the messaging send endpoint. The application messages are packaged in a so-called **envelope**, which contains transport layer metadata. The UCRM MUST perform the following checks and return an appropriate error object in case of an error:

1. Check whether the KT is authorized to send the message (compare the source field with the transferred OAuth token).
2. Check whether the addressed KT (destinations field) is known.
3. Check whether the application message contained (payload field) references a known application in a known application version and is valid according to the message schema.
4. Check whether the addressed KT (destinations field) supports the receipt of the application message according to its KT registration entry.

The UCRM does NOT check message signatures for messages sent by clients; this is the responsibility of the message recipient.

If the UCRM reports an unknown destination, the client should use the /info endpoint to check whether the UCRM is in a normal operating state or is currently performing the initial retrieval of KT registration data from connected UCRMs. In this case, you should wait until the UCRM reports a normal operating state.

Delivery acknowledgments

To ensure reliable delivery, a client can set a timeout value for message delivery (timeout field) and request an explicit acknowledgment of receipt (ack field). In these cases, the UCRM transmits the following to the client

either a negative acknowledgment (if a timeout has occurred or an error has occurred during message forwarding within the transport layer) or a positive acknowledgment (if this was requested by the ack field and the recipient has confirmed receipt of the message via the messaging commit endpoint). The

message_delivery_status message from the transport-layer-messages app is on mission. The following specifications apply to these status notifications:

1. message_delivery_status messages may ONLY be sent by UCRMs, not by clients. This must be ensured by the UCRM.
2. A UCRM should report a MAXIMUM of one message_delivery_status message to the KT for a sent message. However, since the transmission of the positive delivery acknowledgment to the sending UCRM may overlap with the generation of the timeout, it is still possible for a KT to receive multiple message_delivery_status messages.

3. The responsibility for sending a negative timeout acknowledgment message ALWAYS lies with the UCRM to which the sender is connected. This ensures that timeout messages are also generated if transmission problems occur with within the transport layer.

Repeated message sending

If a KT wants to resend a message (e.g., because a timeout was reported for the first transmission attempt), the same **messageId** MUST be used for the resent message. This ensures that the duplicate message can be filtered out by the receiving KT based on the **messageId**.

4.1.6. /messaging/receive - HTTP POST

The messaging receive endpoint can be used to retrieve messages addressed to the KT (or a list of OIDs in the **destinations** field). This operation is idempotent, meaning it can be repeated multiple times with the same result.
To prevent repeated receipt of a message, the KT MUST confirm the successfully received messages via the messaging commit endpoint before the next message retrieval.

The UCRM MUST check whether the client is authorized to retrieve the transferred OIDs (by comparing them with the transferred OAuth token).

For each message received, the UCRM MUST return a monotonically increasing sequence number in the **sequenceId** field, which does NOT change for an individual message in the event of multiple retrievals.

Long polling

Since the messaging receive endpoint must be retrieved periodically, long polling is provided for this endpoint to conserve network resources and ensure the lowest possible delivery delays. This MUST be supported by the UCRM. The following loop must be used for message queries using long polling:

1. Query messages specifying the maximum number of messages nMax (field **maxMessages**)
2. If messages have been reported, process messages
3. If messages have been reported, confirm message receipt

Important: Program failures between steps 2 and 3 may result in the same messages being received repeatedly. The client logic MUST therefore be able to handle duplicate messages. This can be achieved by taking into account the uniqueness of the message ID (**messageId** field).

By using long polling, you can move directly from step 3 to step 1 without a pause. The following specifications apply to long polling:

1. The maximum delay (dMax) for the response is 30 seconds.
2. If there is at least one unconfirmed message for at least one of the requested destinations (field **destinations**), the request is answered immediately.
3. Otherwise, the response to the client request is delayed until one of the following conditions is met (the conditions must be checked in the order listed), at which point the request is answered without further delay:
    1. At least one unconfirmed message is present for at least one of the requested destinations (**destinations** field).
    2. The maximum delay (dMax) for the response has been reached.
4. If problems arise during long polling that cannot be solved by other means (e.g., by extending TCP timeouts on the client side), the client can use the **maxDelay** field to specify a shortened dMax, which is then used by the UCRM instead of the dMax specified in 1.

Availability status

The periodic retrieval of the messaging receive endpoint also serves as an indicator for the UCRM of the current availability of the client. If a client does not retrieve the messaging send endpoint for 60 seconds (availability timeout), the UCRM MUST
Set the KT registration availability status (**status** field) to "offline." As soon as another request comes in, the UCRM MUST

set the availability status to "online." This allows other KTs to retrieve the current availability status via KT registration. By setting the availability status timeout (60 seconds) to twice the maximum long polling delay (30 seconds), it is ensured that both functions work together harmoniously. In order to appear available from the perspective of the switching level, a client MUST therefore perform a permanent query of messages (see the loop described in the "Long Polling" chapter).

4.1.7. /messaging/commit - HTTP POST

The messaging commit endpoint can be used to confirm the successful receipt of messages for a single destination. A reference sequence number is specified for this purpose (**sequenceId** field). The UCRM then removes all messages with a message sequence number <= reference sequence number from the list of unconfirmed messages and, depending on the configuration of the ack field, triggers the transmission of a delivery confirmation (**message-delivery-status**). If a client retrieves messages for different OIDs, a separate call to the messaging commit endpoint MUST be made for each message.

This operation is idempotent, meaning it can be repeated multiple times with the same result.

The UCRM MUST check whether the client is authorized to confirm messages to the transferred OIDs (**destination** field) (by comparing with the transferred OAuth token).

4.2. Peer-to-Peer API (P2P API)

The P2P API is used for networking within the transport layer, i.e., between **UCRMs**. Each participating UCRM provides the API itself on the one hand and connects to the APIs offered by remote UCRMs on the other. The API endpoints are presented below:

4.2.1. /token - HTTP GET

JSON Web Tokens (JWT) are retrieved via the token endpoint and used for authorization at all other endpoints
. For this purpose, a user ID and a secret are transferred via HTTP header. It should be noted that, according to RFC 7519, is represented as an exp claim as NumericDate (i.e., as the seconds since 1970-01-01T00:00:00Z UTC), see RFC 7519 Chapter 2.

4.2.2. /info - HTTP GET

The info endpoint can be used to retrieve general information about the UCRM, such as the UCRI2 transport layer version used, UCRM manufacturer, product version, and operating status of the UCRM.

4.2.3. /registry - HTTP GET

The registry endpoint allows remote UCRMs to retrieve information about all communication participants directly connected to the called UCRM (including the UCRMs themselves). In this case, the UCRM MUST return only the local KT, but not other KT connected via remote UCRMs.

If individual transmitted KT registration records are incorrect, the UCRM MUST reject them.

Since the environment is dynamic, i.e., new KTs can be added and existing KTs can be removed, KT registry queries should be performed regularly (at least once per hour, at most every 5 minutes). In addition, a UCRM should retrieve the KT registration data of all known other UCRMs at startup. During this phase, a UCRM MUST set the status value returned at the info endpoint to status 1 (for both the P2P and client APIs).

Direct transmission of availability status messages

If a UCRM detects that a client's availability status has changed, the KT registration status field is updated (see subchapter "Client API/Availability Status"). Remote UCRMs are only notified of this change
once they perform the next periodic retrieval of the KT registration. In order to communicate the status change directly and without

delay, the UCRM MUST therefore also send a participant_availability_update message (from the transport_layer_messages app) to all remote UCRMs after detecting the status change.

If a UCRM receives such a participant_availability_update message, it MUST adjust the KT registration status of the specified KT to the transmitted value. The message itself MUST NOT be forwarded to the connected clients.

4.2.4. /messaging/send - HTTP POST

Messages are forwarded to remote UCRMs via the messaging send endpoint. The following specifications apply:

1. If the remote UCRM rejects the message and "none" was not specified in the ack field, the sending UCRM MUST send a negative delivery status message to the KT that originally sent the message, with the reported error included in the cause field.
2. The receiving UCRM MUST check whether the message is addressed to one of the KTs connected to the UCRM or to the UCRM itself; if this is not the case, the message MUST be rejected.
3. The receiving UCRM MUST check whether the application message contained (payload field) references a known application in a known application version and is valid according to the message schema.
4. The receiving UCRM MUST check whether the addressed KT (destination field) supports the receipt of the application message in accordance with its KT registration entry.

4.3. Error handling

If an error is returned in response to a request, it ALWAYS contains a numeric, UCRI2-specific error code. This error code makes it possible to determine the cause of the error more precisely in addition to the rough error classification via HTTP status code. A fixed error code ID is assigned to each error code, e.g., for error code 460, the error code ID REQUEST_INVALID_PER_CLIENT_TRANSPORT_SPEC (see below).

Error codes other than those listed below must NOT be returned.

Unless explicitly stated otherwise, all error codes can be returned by both the client and P2P interfaces.

In addition to the error code, a human-readable error message is ALWAYS included in the reason field. However, the additional transmission of a detailed error cause in the message field is optional.

For each error code, the situations in which this error code is returned are shown here. The error codes are sorted according to the HTTP status codes in whose context they occur. Error codes may ONLY be transmitted in the context of the listed HTTP status code. An exception to this is the transmission of a NAK delivery error as part of the message_delivery_status message in the transport_layer_messages app, where the error code reported by a remote UCRM via the P2P interface is forwarded to the client.

4.3.1. Error codes for responses with HTTP status code 400

- `460 (REQUEST_INVALID_PER_CLIENT_TRANSPORT_SPEC)` This error code is ONLY returned by the client interface if the request contains valid JSON data but violates the client transport layer schema for this request. The error code is NOT used in mission if an app payload violates the app schema.
  (see `REQUEST_PAYLOAD_INVALID_PER_APP_SPEC`).
- `461 (REQUEST_PAYLOAD_UNKNOWN_APPID)` : This error code is returned if the specified AppId is unknown to the UCRM is unknown.
- `462 (REQUEST_PAYLOAD_UNKNOWN_APPVERSION)` : This error code is returned if the UCRM recognizes the AppId is known to the UCRM, but the specified AppVersion is unknown.

- `463 (REQUEST_PAYLOAD_UNKNOWN_SCHEMAID)` : This error code is returned if the UCRM knows the AppId and AppVersion, but the specified SchemaId (i.e., the message type) is unknown.
- `464 (REQUEST_PAYLOAD_INVALID_PER_APP_SPEC)` : This error code is returned if the request payload in the data field violates the associated app schema.
- `465 (REQUEST_PAYLOAD_INVALID_JSON)` : This error code is returned if the transferred data is not valid JSON.
- `466 (REQUEST_PAYLOAD_UNSUPPORTED_APPID_OR_APPVERSION)` : This error code is returned if the appId and appVersion are known to the UCRM but are not listed as supported in the supportedApps of the destination.
- `468 (REQUEST_PAYLOAD_UNSUPPORTED_MESSAGE)` : This error code is returned if the appId and appVersion are known to the UCRM and are also listed as supported in the supportedApps of the destination, but the schemaId (i.e., the message type) is listed in the corresponding supportedApps entry in the unsupportedMessages list.
- `467 (REQUEST_PAYLOAD_FORBIDDEN_APPID)` : This error code is ONLY returned by the client interface if the message sent belongs to the transport_layer_messages app (messages from this app may only be sent by UCRMs).
- `470 (REQUEST_UNKNOWN_DESTINATION_ID)` : This error code is returned if the specified destination is not known to the UCRM.
- `478 (REQUEST_OID_FORBIDDEN)` : This error code is returned if the OAuth token passed identified user does not have access to the specified OID. The specified OID is the OID:
  - in the sender field (for the /messaging/sent endpoint)
  - in the destinations field (for the /messaging/receive endpoint)
  - in the destination field (for the /messaging/commit endpoint)
- `479 (REQUEST_WRONG_SIGNATURE)` This error code is ONLY returned by the P2P interface if the message sent is a signed message from the transport_layer_messages app (and the sending UCRM has not specified via transmitsUnsignedMessages in its KT registration entry that it does not sign its messages). On the client interface and generally for messages sent by clients, NO signature verification takes place at the transport layer.
- `480 (REQUEST_INVALID_PER_P2P_TRANSPORT_SPEC)` This error code is ONLY returned by the P2P interface if the request contains valid JSON data but violates the P2P transport layer schema for this request. The error code is NOT used in mission if an app payload violates the app schema (see REQUEST_PAYLOAD_INVALID_PER_APP_SPEC).

4.3.2. Error codes for responses with HTTP status code 401

- `475 (REQUEST_UNAUTHORIZED)` : This error code is returned if the OAuth token provided is missing, invalid, or expired. The error code is also returned by the /token endpoint if the credentials provided are incorrect.

4.3.3. Error codes for responses with HTTP status code 500

- `491 (REQUEST_INTERNAL_ERROR)` This error code is returned if an internal error has occurred.

5. Applications

The specific UCRI2 app specifications are provided separately. The document "UCRI2 Apps at a Glance" provides an overview of the various apps.

In addition, there is a PDF file with textual app documentation for each app in each version, as well as a JSON schema file for each app message that specifies the message structure.

6. System integration

The UCRI2 specification provides a standard framework for integrating control center software from different manufacturers. It helps to overcome compatibility issues and thus ensures interoperability in complex IT landscapes. At the same time, the switching level remains open for the seamless integration of existing system solutions into new communication systems. This makes it possible to continue using existing infrastructures, thereby reducing development costs and times. The result is a sustainable and efficient system landscape that guarantees both innovation and continuity.

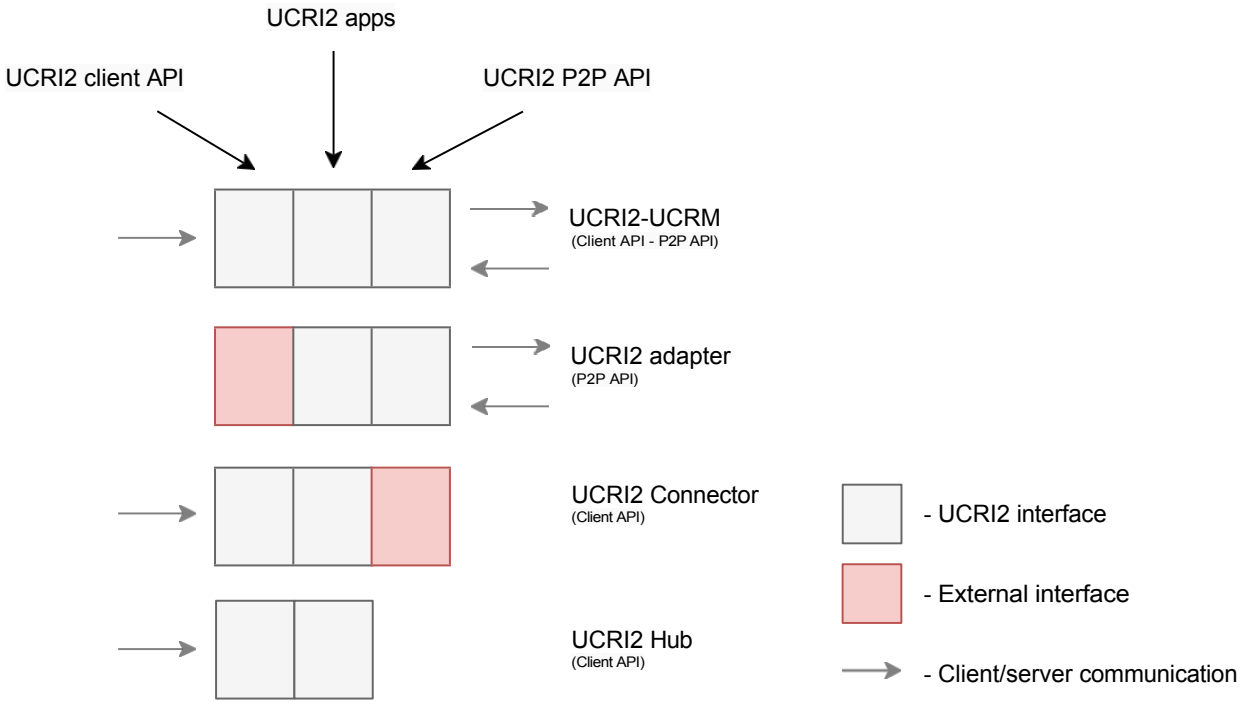
This chapter presents integration patterns and associated integration scenarios.

6.1. Integration patterns

The central element of the UCRI2 specification is the UCRM—a system component that is described by the following interfaces:

- The **client API** is provided by **the UCRM** and consumed by **clients** (communication participants).
- The **P2P API** is offered by **UCRMs** that want to establish a connection with other **UCRMs** based on a peer-to-peer topology.

Depending on the system landscape, system integrators need specialized UCRM modules that support different integration scenarios:



UCRI2-UCRM

The classic UCRI2-UCRM module is a universal component that is used as an intermediary in a standard UCRI2 network. The module connects one or more control centers connected via the UCRI2 Client API with other control centers that are connected via

a third-party UCRI2-UCRM module. The UCRI2 P2P protocol is used between UCRMs to transmit messages and exchange other UCRI2-specific data.

UCRI2 adapter

The UCRI2 adapter is used to integrate an existing control center communication system into a UCRI2 network. The module provides an interface that the existing system expects. The UCRI2 adapter acts as an intermediary that translates a third-party communication protocol, including mapping between addressing schemes, into the UCRI2 P2P protocol.

UCRI2 Connector

The UCRI2 connector connects a control center connected via the UCRI2 client API to a third-party messaging system. The UCRI2 connector encapsulates specific third-party protocols, data flows, and communication mechanisms, including mapping between addressing schemes, to enable integration via an existing third-party switching system.

UCRI2 Hub

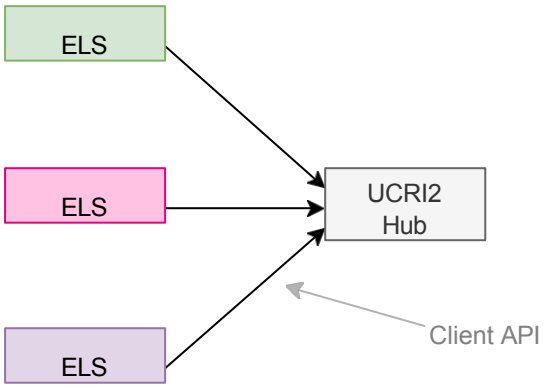
The UCRI2 Hub serves as a central component that acts as an intermediary between several control centers connected via the UCRI2 Client API. All messages flow through this central point, decoupling communication between the participants. The UCRI2 Hub does not support switching layer protocols and therefore cannot be connected to other UCRMs.

6.2. Integration scenarios

The following section presents various integration scenarios using specialized UCRM modules.

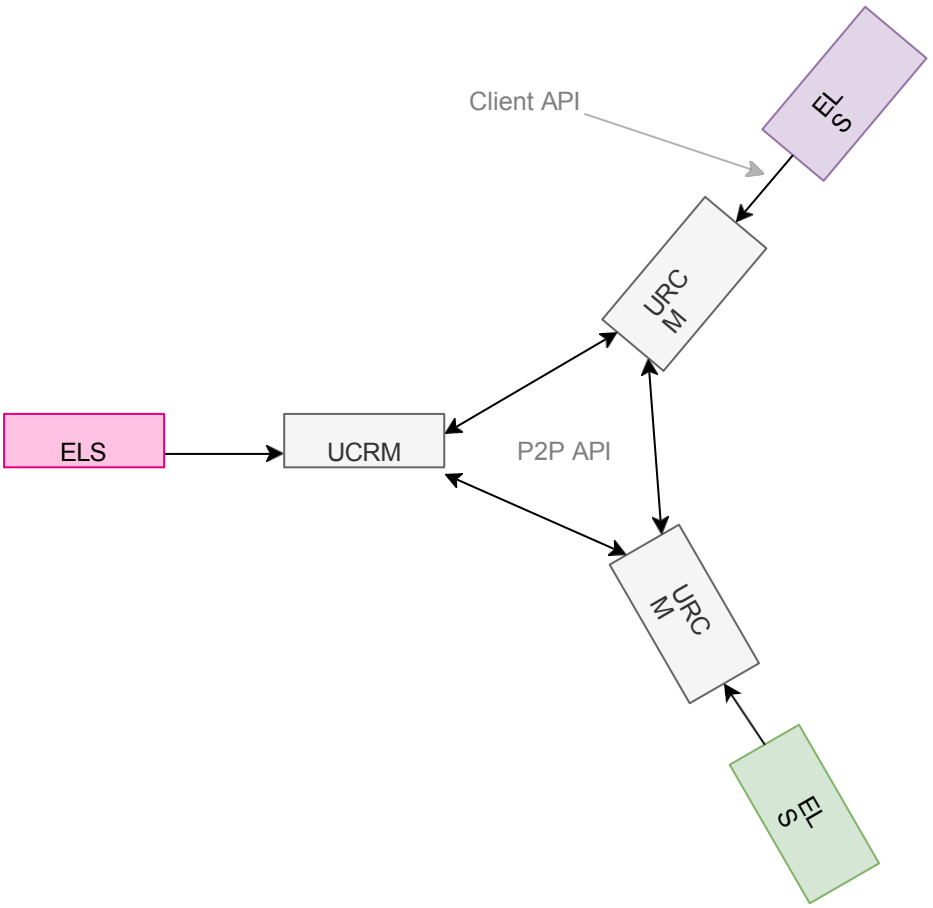
A UCRI2 hub as an intermediary

In the simplest case, control centers are connected via a central UCRI2 hub. The ELS software is connected to this module via the UCRI2 client API. This allows decoupled ELS systems to communicate with each other. The UCRI2 hub does not support switching layer protocols and therefore cannot be connected to other UCRMs.



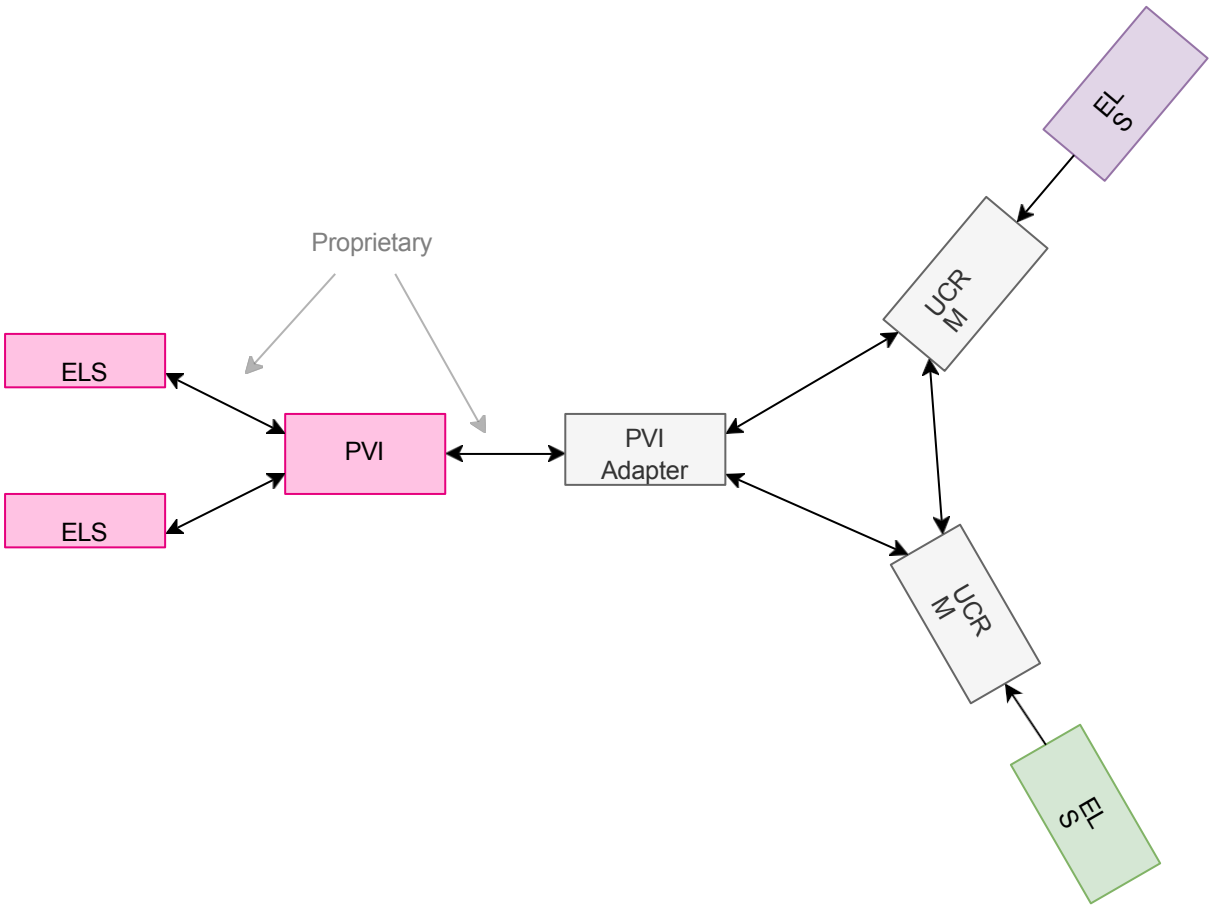
The standard UCRI2 network

In a standard UCRI2 network, control centers are connected via classic UCRI2-UCRM modules. Typically, a UCRI2-UCRM module is established in the infrastructure of a control center. The ELS software is connected to this module via the UCRI2 client API. The UCRI2-UCRM modules of different communication participants are connected to each other via the UCRI2 P2P API.



Connection to network with PVI

In this scenario, an existing control center communication system is integrated into a UCRI2 network. The connection is established via a PVI adapter—a specialized UCRI2 adapter that supports the communication interface of a proprietary switching instance. On the other hand, the PVI adapter speaks the UCRI2 P2P protocol and can thus be incorporated into the P2P topology of the network. The PVI adapter translates the PVI communication protocol into the UCRI2 P2P protocol, including mapping between addressing schemes.



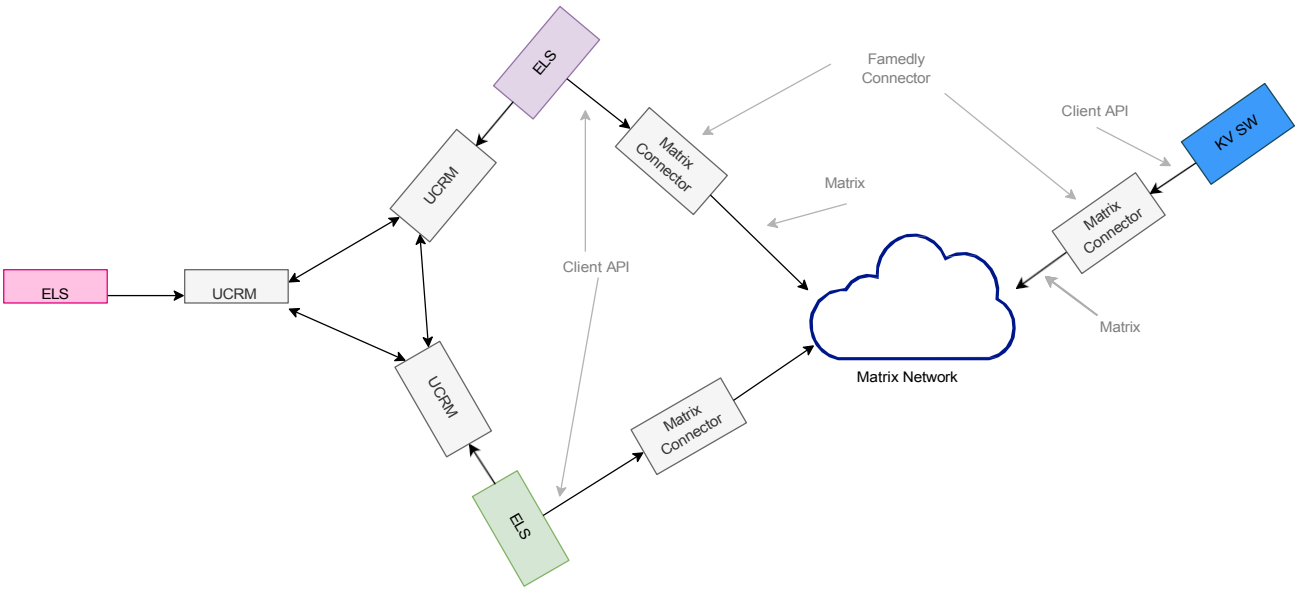
Network with connection to KV network without KV adapter

An important use case for the UCRI2 specification is communication between 110/112 and KV control centers.

The starting point for this integration scenario is an existing standard UCRI2 network and a requirement to connect a specific control center from the network to a KV control center. The current development, according to which KVs must communicate via a standardized switching architecture based on a matrix infrastructure, should be taken into account.

For integration between control centers, a matrix connector—a UCRI2 connector for connection to a matrix infrastructure—is used on both sides. Both ELS and KV software use the UCRI2 client API of their own matrix connector. Messages are transmitted via the matrix protocol. The Matrix connector supports mapping between UCRI2 and Matrix addressing schemes.

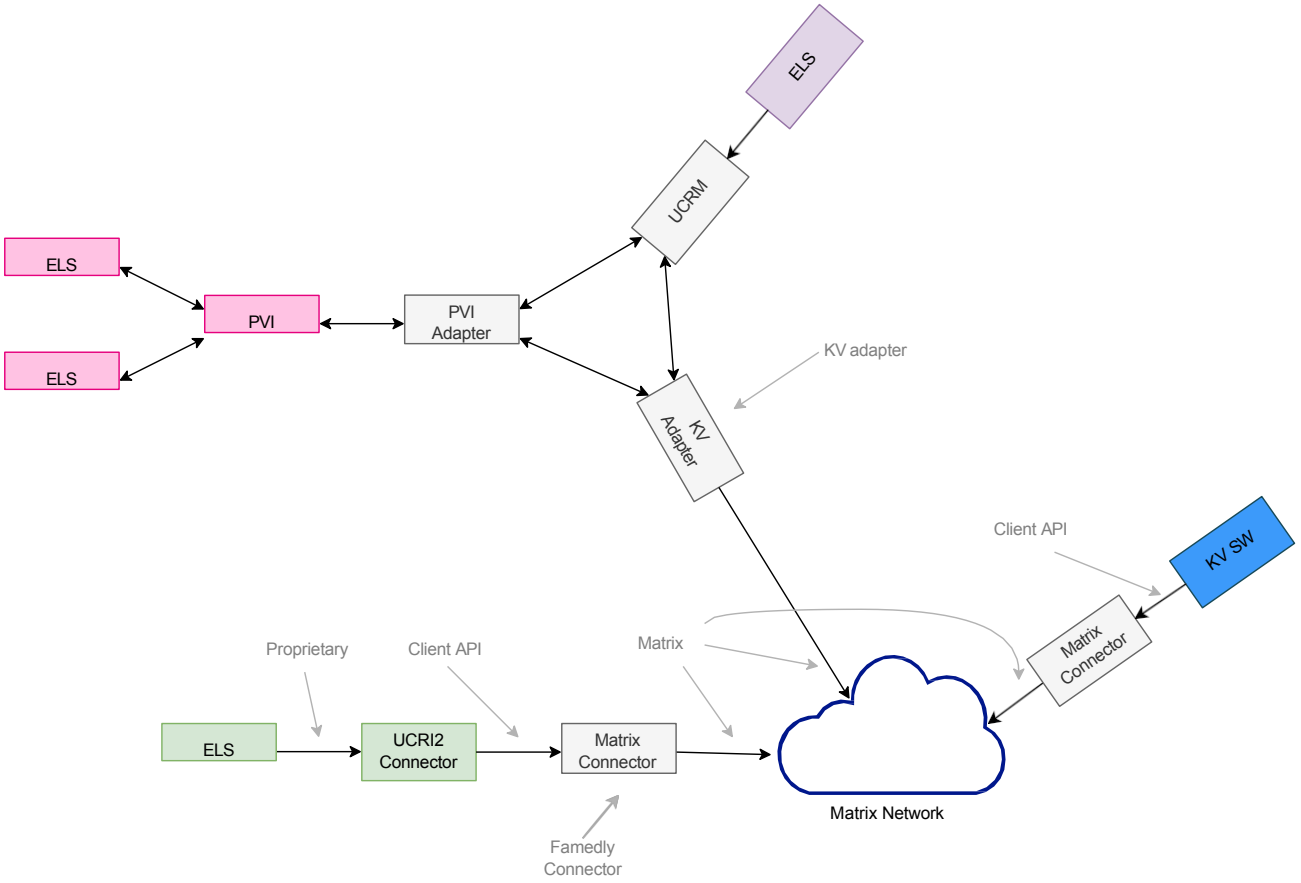
One problem with this integration scheme is that each control center that is to be integrated with KVs requires its own Matrix Connector in addition to the UCRM module. The solution to this problem is to connect the entire UCRI2 network to the Matrix infrastructure, see next integration scenario.



Network with connection to KV network with KV adapter

In this scenario, the entire UCRI2 network is connected to the Matrix infrastructure using a Matrix adapter. This allows all ELSs participating in the network to communicate with KVs without having to use their own Matrix connector.

The Matrix adapter supports the UCRI2 P2P protocol and is established as part of an existing UCRI2 P2P network. On the other hand, the Matrix adapter supports the Matrix protocol and can therefore communicate with UCRI2 Matrix connectors of other participants in a Matrix infrastructure. In this way, UCRI2 domains based on different switching technologies are organized into a higher-level UCRI2 communication network in which all participants can communicate transparently with each other using standardized UCRI2 applications.


ELS
UCRM
ELS
PVI
PVI Adapter
ELS
KV adapter
KV Adapter
Client API
KV SW
Matrix
Matrix Connector
Proprietary
Client API
ELS
UCRI2 Connector
Matrix Connector
Matrix Network
Famedly Connector